

Graduate Algorithms

Problems on Divide and Conquer

27 August 2026

A lot of time and effort goes into making miracles. So it's easy to take them for granted. Same goes for healing sick people or flying. The moment they become ordinary and boring, they stop being miracles or magic. (But, if you look closely, they are still miracles)

— Hyeon A Cho, Your Letter

☰ Counting Inversions

Given an (1-indexed) array A of n distinct integers, Give an algorithm to count the number of inversions in $O(n \log n)$

An inversion is a pair of i, j such that

- $1 \leq i < j \leq n$
- $A[i] > A[j]$

Hint: Consider a function `def inversions(A: list[int]) -> tuple[int, list[int]]` where we are returning both the number of inversions and sorted list A .

☰ Strassen's Algorithm

Given two $n \times n$ matrices A and B , compute their product AB in time $O(n^{\log_2 7})$

Assuming $m \geq n$, can you find an $O(m^2 n^{0.81})$ algorithm to multiply A a $m \times n$ and B a $n \times m$ matrix using the above a subroutine. (Hint: Multiply $\lceil \frac{m}{n} \rceil^2$ pairs of order- n matrices)

Assuming $m \geq n \geq t$, Can you find an $O(mnt^{0.81})$ algorithm to multiply A a $m \times n$ and B a $n \times t$ matrix using the above a subroutine. (Hint: Multiply pairs of $t \times t$ matrices)

☰ Merging Arrays

Let $A_1, A_2, \dots, A_k$ be k arrays, each of which has been sorted.

These arrays are mutually disjoint, namely, no integer can appear in more than one array.

Design an algorithm to merge the k arrays into one sorted array in $O(n \log k)$ time, where n is the total length of the k arrays.

Note: these arrays may have different lengths.

☰ Max Weight Subarray

Let A be an array of n integers (A is not necessarily sorted). Each integer in A may be positive or negative.

Given i, j satisfying $1 \leq i \leq j \leq n$,

Define sub-array $A[i : j]$ as the sequence $(A[i], A[i + 1], \dots, A[j])$, and the weight of $A[i : j]$ as $A[i] + A[i + 1] + \dots + A[j]$.

For example, consider $A = (13, -3, -25, 20, -3, -16, -23, 18)$; $A[1 : 4]$ has weight 5, while $A[2 : 4]$ has weight -8 .

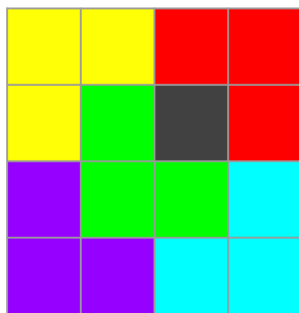
1. Give an algorithm to find a sub-array of with the largest weight, among all sub-arrays $A[i : j]$ with $j = n$. Your algorithm must finish in $O(n)$ time.
2. Give an algorithm to find a sub-array with the largest weight in $O(n \log n)$ time (among all the possible sub-arrays).

☰ L Tiles

Given a grid of $2^N \times 2^N$, all cells are initially empty except one of the cells. Let the blocked cell be B .

Write an algorithm to figure out a way to fill the grid with L shapes (without overlapping other tiles or B) or report “NO” if that is not possible. Your algorithm should run in $O(4^N)$.

The $N = 2, B = (3, 2)$ case is given below.



Hint : We never have to return “NO”.

☰ Big Mod

Given p, e, b , compute $p^e \bmod b$ in $O(\log e)$ time.

☰ DeadShot (DS Algo Mock-Midsem 2026)

To celebrate the end of midsems, a group of students head to the CLAB to play Deadshot.io. There are n players participating, and the i th player has a skill score of $A[i]$.

They want to split into two teams for a match. They agree to the following rules:

- Team Kabir must have a total skill score greater than or equal to Team Piyush.
- The difference between the total skill scores of the two teams should be as small as possible.

Write a function `balanced_teams(A)` that returns the minimum possible difference between the total skill scores of Team Kabir and Team Piyush under these rules in $O(n2^{\frac{n}{2}})$ time.

Input	Output	Explanation
$A = [5, 4, 7, 6]$	0	Team Kabir gets $5 + 6 = 11$ and Team Piyush gets $4 + 7 = 11$ skill.
$A = [10, 1, 1]$	8	Team Kabir gets 10 skill Team Piyush gets $1 + 1 = 2$ skill.
$A = [3, 4, 2, 2]$	1	Team Kabir gets $4 + 2 = 6$ skill and Team Piyush gets $3 + 2 = 5$ skill.

Hint: The intended solution uses $O(2^{\frac{n}{2}})$ space as we need **two** arrays of that length...(This should ring some bells)

- Write pseudocode implementing `balanced_teams(A)`. You are not allowed to import libraries or use any language specific features.
- Argue that your algorithm runs in $O(n2^{\frac{n}{2}})$.
- Argue the correctness of your implementation.